

STAJDESK

48 GÜNLÜK DERS KİTABI

Sıfırdan öğren • Yavaş yavaş uygula • Nedenini anla • Kendi cümlele anlat

BU KİTAP NE İÇİN?

Kaynak programdaki 16 hafta ve 48 günün tamamı esas alınmıştır. Her gün için konu anlatımı, günlük hayat benzetmesi, örnek kod, satır satır düşünme, uygulama görevleri ve gün sonu kontrolü eklenmiştir. Amaç ezber değil, neden-sonuç ilişkisi kurarak öğrenmektir.

Teknik doğruluk için kritik noktalarda resmi dokümantasyonla karşılaştırılmış açıklamalar kullanılmıştır. Teknoloji sürümleri değişebileceği için kurulum sırasında güncel resmi dokümantasyon kontrol edilmelidir.

Bu kitabı nasıl çalışmalısın?

Bir günün tamamını okumak zorunda değilsin. Önce kavramı oku, sonra örnek kodu kendin yaz, kod üzerinde küçük bir değişiklik yap ve son olarak konuyu kendi cümlele anlat. Takıldığın yerde LOG.md'ye not düş.

1 • Anla	2 • Uygula
Kavramın hangi problemi çözdüğünü bul.	Küçük örneği çalıştır; sonra kendin değiştir.
3 • Anlat	4 • Kaydet
Mentora anlatır gibi yüksek sesle açıkla.	Görevi tamamla ve anlamlı commit bırak.

ALTIN KURAL

Bir kod parçası çalışıyor diye öğrenilmiş sayılmaz. “Bu satır neden burada?” sorusuna cevap verebildiğinde konu oturmaya başlar.

İçindekiler

Hafta	Konu	Günler
1	Başlangıç, Git ve Web'in Çalışma Mantığı	Gün 1 - 3
2	Docker ile Tanışma	Gün 4 - 6
3	C# Temelleri	Gün 7 - 9
4	Nesne Yönelimli Programlama (OOP)	Gün 10 - 12
5	ASP.NET Core Web API	Gün 13 - 15
6	İlişkisel Veritabanı ve SQL	Gün 16 - 18
7	Entity Framework Core ile API'yi Veritabanına Bağlamak	Gün 19 - 21
8	Doğrulama, Hata Yönetimi ve Kimlik Doğrulama	Gün 22 - 24
9	Frontend Temelleri ve Vue.js'e Giriş	Gün 25 - 27
10	Vue Router, Pinia ve API Bağlantısı	Gün 28 - 30
11	Uygulama Ekranları	Gün 31 - 33
12	Redis ile Önbellekleme ve Hız Sınırlama	Gün 34 - 36
13	RabbitMQ ile Asenkron Mesajlaşma	Gün 37 - 39
14	Tüm Sistemin Docker ile Çalıştırılması	Gün 40 - 42
15	Kalite: Test, Loglama ve Dokümantasyon	Gün 43 - 45
16	Son Dokunuşlar ve Final Sunumu	Gün 46 - 48

BÜYÜK RESİM

Konsol → Web API → PostgreSQL → EF Core → Güvenlik → Vue → Redis → RabbitMQ → Docker Compose → Test ve Dokümantasyon → Final. Her hafta bir öncekinin üzerine yeni bir katman ekler.

Teknik doğruluk: en çok karıştırılan 10 nokta

- Image çalışan programın kendisi değil, container oluşturmak için kullanılan paketlenmiş şablondur.
- EXPOSE Dockerfile içinde port niyetini belirtir; host'a port yayınlamak için `-p`/Compose ports` gibi bir yayınlama gerekir.
- Frontend validation kullanıcı deneyimidir; backend validation ve authorization güvenlik sınırıdır.
- JWT payload'ı normalde gizli değildir. İmzalı olması, verinin bütünlüğünü doğrulamaya yardım eder; parolayı payload'a koymak doğru değildir.
- 401 kimlik doğrulama eksik/geçersiz; 403 kimliği doğrulanmış ama yetkisiz kullanıcı için tipik ayrımdır.
- `depends_on` servisler arası başlangıç bağımlılığını ifade eder; uygulamanın gerçekten hazır olması için `healthcheck/readiness` stratejisi gerekir.
- RabbitMQ'da dayanıklılık tek bir ayarla garantilenmez; `queue/message durability`, `acknowledgements` ve `publisher confirms` gibi mekanizmalar ayrı sorumluluklardır.
- Redis cache olarak source of truth olmak zorunda değildir; `stale data` ve `cache invalidation` ayrıca tasarlanır.
- EF Core migration'ları geliştirici ortamında pratik olsa da production deployment stratejisi ayrıca seçilip gözden geçirilmelidir.
- `.env` içindeki değerler istemci tarafına bundle oluyorsa secret sayılmaz; `VITE_` ile yayımlanan client-side config gizli anahtar olarak kullanılmamalıdır.`

ÖĞRENMEYİ KOLAYLAŞTIRAN HATIRLATMA

Doğru cevap sadece “ne?” değildir. İyi teknik açıklama üç parçadan oluşur: ne işe yarar → neden kullanılır → hangi sınırlaması vardır.

Tanışma ve ortam kurulumu

BUGÜNÜN HEDEFİ

Geliştirme ortamını kurmak, Git ile çalışmaya başlamak ve web isteğinin arka planda nasıl işlendiğini kavramak.

1. Önce konuyu anla

Bugün bir yazılım projesinde çalışmak için gereken temel araçları tanıyorsun. Her araç farklı bir sorumluluğu üstlenir; önemli olan komutları ezberlemek değil, hangi problemi çözdüklerini bilmektir.

GÜNLÜK HAYATTAN BENZETME

Bir marangoz atölyesi düşün: VS Code tezgâh, Git kayıt defteri, .NET çalışma motoru, Node.js frontend araç kutusu, Docker ise taşınabilir çalışma kabini gibidir.

2. Kaynak programda ne öğrenilecek?

- Stajın amacı, StajDesk projesinin tanıtımı ve 16
- Bir yazılım ekibi nasıl çalışır: görev takibi (Jira),
- Kullanılacak araçlar ve ne işe yaradıkları

3. Uygulama / ödev

- ☐ VS Code, Git, .NET SDK, Node.js (LTS) ve Docker Desktop
- ☐ dotnet --version, node -v, git --version, docker--version çıktılarını not ediniz
- ☐ Masaüstünde stajdesk klasörü açıp içine LOG.md (öğrenme

4. Örnek üzerinden yavaşça ilerle

Gün 1 • örnek

```
dotnet --version
node -v
git --version
docker --version
```

5. Bu kodun fikri nedir?

- Versiyon komutları araçların PATH üzerinde ve çalışır durumda olduğunu kontrol etmeye yarar.
- LOG.md günlük ilerlemeyi görünür tutar.
- Araç kurulumundan sonra küçük bir test yapmak, daha sonra çıkacak hataların kaynağını daraltır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Tanışma ve ortam kurulumu ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Tüm araçlar kurulu ve çalışıyor; LOG.md'de ilk günlük kaydı var.

Terminal ve Git temelleri

BUGÜNÜN HEDEFİ

Geliştirme ortamını kurmak, Git ile çalışmaya başlamak ve web isteğinin arka planda nasıl işlendiğini kavramak.

1. Önce konuyu anla

Terminal, bilgisayarın dosya sistemi ve Git ile doğrudan konuştuğun çalışma alanıdır. Git'in en önemli fikri, değişiklikleri anlamlı küçük kayıtlar halinde takip etmektir.

GÜNLÜK HAYATTAN BENZETME

Bir projenin her hâlini final-final2-finalson diye kopyalamak yerine düzenli sürüm kayıtları tuttuğunu düşün.

2. Kaynak programda ne öğrenilecek?

- Terminal komutları: cd, ls/dir, mkdir, dosya
- Git nedir, neden kullanılır: repo, commit, branch,
- İyi bir commit mesajı nasıl yazılır

3. Uygulama / ödev

- ☐ Şirket Git sunucusunda stajdesk deposunu oluşturup
- ☐ README.md yazınız (kendinizi ve projeyi tanıtın), en az 5
- ☐ Bir deneme branch'i açıp değişiklik yapınız ve ana dala merge

4. Örnek üzerinden yavaşça ilerle

Gün 2 • örnek

```
git status
git switch -c feature/deneme
git add .
git commit -m "docs: README ekle"
git switch main
git merge feature/deneme
git push origin main
```

5. Bu kodun fikri nedir?

- commit yerel Git tarihçesine kayıt ekler; push başka bir depoya gönderir.
- branch ayrı bir çalışma çizgisidir.
- merge bir branch'teki değişiklikleri başka bir çizgiyle birleştirir.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Terminal ve Git temelleri ne işe yarıyor?" sorusuna kendi cümlemle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Depo linki mentorla paylaşıldı; commit geçmişi düzenli.

Web nasıl çalışır? (HTTP ve JSON)

BUGÜNÜN HEDEFİ

Geliştirme ortamını kurmak, Git ile çalışmaya başlamak ve web isteğinin arka planda nasıl işlendiğini kavramak.

1. Önce konuyu anla

Web uygulamasını anlamanın anahtarı, istemci ile sunucunun mesajlaşmasını görmektir. URL nereye gideceğimizi, HTTP metodu ne yapmak istediğimizi, durum kodu sonucu ve JSON ise taşınan veriyi ifade eder.

GÜNLÜK HAYATTAN BENZETME

Restoranda garson istemci, mutfak sunucu, sipariş HTTP isteği, hazırlanan yemek HTTP yanıtı gibidir.

2. Kaynak programda ne öğrenilecek?

- İstemci-sunucu modeli, URL yapısı, DNS'e kısa
- HTTP metotları (GET, POST, PUT, DELETE) ve
- JSON veri formatı; tarayıcı geliştirici araçlarında

3. Uygulama / ödev

- ☐ 3 farklı sitede Network sekmesinden istekleri inceleyip
- ☐ Postman veya Bruno ile jsonplaceholder.typicode.com
- ☐ Cuma değerlendirmesi: haftanın özetini mentora 5 dakikada

4. Örnek üzerinden yavaşça ilerle

Gün 3 • örnek

```
GET /api/tickets
POST /api/tickets

{
  "title": "Yeni talep",
  "description": "Açıklama"
}
```

5. Bu kodun fikri nedir?

- GET okuma için, POST yeni kaynak/işlem için, PUT güncelleme için, DELETE silme için sık kullanılır.
- 404 kaynağın bulunamadığını, 500 ise sunucuda beklenmeyen hata olduğunu anlatır.
- 401 kimlik doğrulama eksik/geçersiz; 403 ise kimliği bilinen kullanıcının yetkisiz olduğunu anlatır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Web nasıl çalışır? (HTTP ve JSON) ne işe yarıyor?” sorusuna kendi cümlemle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

HTTP istek/yanıt akışını kendi cümlelerinizle anlatabiliyorsunuz.

6. Kavrama soruları

- 1 Tarayıcıya bir adres yazıp Enter'a bastığınızda neler olur?
- 2 Commit ile push arasındaki fark nedir?
- 3 404 ile 500 durum kodu arasındaki fark nedir? Hangisi kimin hatasıdır? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 7 FAZ 0 · TEMELLER Hafta 2 | Docker ile Tanışma Haftanın hedefi: Konteyner mantığını anlamak; image, container, volume ve docker compose kavramlarını uygulamalı öğrenmek.

Konteyner nedir, neden Docker?

BUGÜNÜN HEDEFİ

Konteyner mantığını; image, container, volume ve Docker Compose'u uygulamalı öğrenmek.

1. Önce konuyu anla

Docker, uygulama ile bağımlılıklarını daha taşınabilir ve tekrarlanabilir bir çalışma birimi içinde paketlemeye yardımcı olur. Image şablon, container çalışan örnektir; port mapping host ile container arasındaki erişim yolunu kurar.

GÜNLÜK HAYATTAN BENZETME

Image bir kalıp, container o kalıptan çıkarılmış gerçek ürün gibidir.

2. Kaynak programda ne öğrenilecek?

- "Benim bilgisayarımda çalışıyordu" problemi ve
- Image ile container farkı; Docker Hub
- Port eşleme (port mapping) ve konteyner

3. Uygulama / ödev

- ☐ docker run hello-world çalıştırınız ve çıktıyı yorumlayınız
- ☐ docker run -d -p 8080:80 nginx ile bir web sunucusu
- ☐ docker ps, docker logs, docker stop, docker rm

4. Örnek üzerinden yavaşça ilerle

Gün 4 • örnek

```
docker run hello-world
docker run -d -p 8080:80 nginx
docker ps
docker logs <id>
docker stop <id>
docker rm <id>
```

5. Bu kodun fikri nedir?

- -d arka planda çalıştırır.
- -p HOST:CONTAINER port eşlemesidir.
- docker ps çalışan container'ları listeler.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Konteyner nedir, neden Docker? ne işe yarıyor?" sorusuna kendi cümlemle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Komut listesi ve ne işe yaradıkları LOG.md'de.

Dockerfile yazmak

BUGÜNÜN HEDEFİ

Konteyner mantığını; image, container, volume ve Docker Compose'u uygulamalı öğrenmek.

1. Önce konuyu anla

Dockerfile, image'ın nasıl üretileceğini adım adım tarif eder. Build zamanında RUN, dosya kopyalamada COPY, taban seçmede FROM ve başlangıç davranışında CMD önemli roller oynar.

GÜNLÜK HAYATTAN BENZETME

Yeni bir bilgisayar kurulum tarifi gibi düşün: tabanı seç, klasörü aç, dosyayı koy, kur ve açılış komutunu tanımla.

2. Kaynak programda ne öğrenilecek?

- Dockerfile komutları: FROM, WORKDIR, COPY, RUN,
- Image katmanları ve build önbellesi
- Volume kavramı: konteyner silinince veri neden

3. Uygulama / ödev

- ☐ Kendinizi tanıtan basit bir HTML sayfası hazırlayınız
- ☐ Bu sayfayı nginx tabanlı bir Dockerfile ile paketleyip docker
- ☐ Dockerfile'ı depoya commit ediniz

4. Örnek üzerinden yavaşça ilerle

Gün 5 • örnek

```
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
COPY index.html .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

5. Bu kodun fikri nedir?

- EXPOSE tek başına host'a port açmaz; beklenen portu belgelemeye yardımcı olur.
- Container katmanındaki geçici verilerin kalıcılığı için volume gibi mekanizmalar gerekir.
- Build cache, önceki adımların değişmemiş sonuçlarını yeniden kullanabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Dockerfile yazmak ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Kendi image'nız çalışıyor ve depoda.

docker compose ve proje mimarisi

BUGÜNÜN HEDEFİ

Konteyner mantığını; image, container, volume ve Docker Compose'u uygulamalı öğrenmek.

1. Önce konuyu anla

Compose, PostgreSQL + Adminer gibi birden fazla servisi tek projede yönetmeyi kolaylaştırır. Servislerin ağ bağlantısı, environment değerleri ve volume'ları aynı yapılandırmada görülebilir.

GÜNLÜK HAYATTAN BENZETME

Bir alışveriş merkezindeki farklı dükkânlar gibi düşün: her biri ayrı ama aynı yapı altında birlikte çalışır.

2. Kaynak programda ne öğrenilecek?

- docker compose ile birden fazla servisi birlikte
- Servisler arası ağ (network), ortam değişkenleri
- StajDesk mimarisi: hangi parça neden var?

3. Uygulama / ödev

- ☐ compose dosyası ile PostgreSQL + Adminer ayağa kaldırınız;
- ☐ Volume tanımlayıp konteyneri silip yeniden başlatınız; verinin
- ☐ Bu dokümandaki mimari şemayı kâğıda kendi elinizle çizip her

4. Örnek üzerinden yavaşça ilerle

Gün 6 • örnek

```
services:
  postgres:
    image: postgres:latest
    volumes:
      - pgdata:/var/lib/postgresql/data
  adminer:
    image: adminer:latest
    ports:
      - "8081:8080"

volumes:
  pgdata:
```

5. Bu kodun fikri nedir?

- Named volume container'ın ömründen bağımsız kalıcı veri alanı sağlar.
- Compose ağında servisler çoğu durumda servis adı üzerinden erişilebilir.
- Gerçek parolaları düz metin olarak paylaşmak yerine secret/config yaklaşımı düşünülmelidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “docker compose ve proje mimarisi ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Çalışan compose dosyası ve el çizimi mimari şema.

6. Kavrama soruları

- 1 Image ile container arasındaki farkı bir benzetmeyle açıklayınız.
- 2 Volume tanımlamasaydık veritabanı konteyneri silindiğinde ne olurdu?
- 3 Neden her geliştiricinin bilgisayarına PostgreSQL kurmak yerine Docker kullanıyoruz? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 8 FAZ 1
· C# VE .NET Hafta 3 | C# Temelleri Haftanın hedefi: C# dilinin temel yapı taşlarını öğrenmek ve basit konsol uygulamaları yazabilmek.

Neden .NET? İlk C# programı

BUGÜNÜN HEDEFİ

C# dilinin temel yapı taşlarını öğrenmek ve basit konsol uygulamaları yazabilmek.

1. Önce konuyu anla

C# statik tipli bir dildir; veri türünü açıkça bilmek daha erken hata yakalamaya yardım eder. dotnet CLI proje oluşturma ve çalıştırma işlerini standardize eder. TryParse ise kullanıcı girdisini güvenli biçimde dönüştürmek için pratiktir.

GÜNLÜK HAYATTAN BENZETME

Çekmeceleri etiketlemek gibi düşün: sayı çekmecesinde sayı, metin çekmecesinde metin tutulur.

2. Kaynak programda ne öğrenilecek?

- Neden .NET: güçlü tip sistemi, yüksek
- dotnet new console, dotnet run; proje
- Değişkenler ve tipler: int, decimal, string,

3. Uygulama / ödev

- ☐ Kullanıcıdan 3 not alıp ortalamasını ve harf notunu yazdıran
- ☐ Hatalı giriş (harf girilmesi) durumunda programın

4. Örnek üzerinden yavaşça ilerle

Gün 7 • örnek

```
Console.Write("Not: ");
if (!int.TryParse(Console.ReadLine(), out int not1))
{
    Console.WriteLine("Geçerli bir sayı gir.");
    return;
}
```

5. Bu kodun fikri nedir?

- int tam sayı, decimal hassas ondalık hesaplar için, string metin, bool doğru/yanlış, DateTime tarih-saat içindir.
- Kullanıcı girdisi güvenilir dış veridir; parse hatası beklenebilir bir durumdur.
- Ortalama hesabında tam sayı bölmesine dikkat et.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Neden .NET? İlk C# programı ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Not hesaplayıcı depoda (/exercises/week03).

Kontrol akışı ve metotlar

BUGÜNÜN HEDEFİ

C# dilinin temel yapı taşlarını öğrenmek ve basit konsol uygulamaları yazabilmek.

1. Önce konuyu anla

Kontrol akışı programın hangi koşulda hangi kodu çalıştıracağını belirler. Metotlar bu davranışları isimli, tekrar kullanılabilir parçalara ayırır.

GÜNLÜK HAYATTAN BENZETME

Tarif: “eğer su kaynadıysa...” if/else, “10 kez karıştır” for döngüsüdür. Metot ise tarifi küçük görevlere böler.

2. Kaynak programda ne öğrenilecek?

- if/else, switch, for, while, foreach
- Metot tanımlama, parametreler, dönüş değeri
- Kodu okunabilir yazmak: anlamlı isimlendirme

3. Uygulama / ödev

- ☐ Sayı tahmin oyunu yazınız (1-100 arası, kaç denemede
- ☐ AsalMi(int sayi) metodu ve 1-100 arası asal sayıları
- ☐ FizzBuzz problemini çözünüz

4. Örnek üzerinden yavaşça ilerle

Gün 8 • örnek

```
static bool AsalMi(int sayi)
{
    if (sayi < 2) return false;
    for (int i = 2; i * i <= sayi; i++)
        if (sayi % i == 0) return false;
    return true;
}
```

5. Bu kodun fikri nedir?

- Metot adı yaptığı işi söylemeli.
- for tekrar sayısı bilinen döngülerde sık kullanılır.
- while koşula bağlı tekrarlar için uygundur.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Kontrol akışı ve metotlar ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

3 alıştırmaya çalışır durumda ve commit'lendi.

Koleksiyonlar ve LINQ

BUGÜNÜN HEDEFİ

C# dilinin temel yapı taşlarını öğrenmek ve basit konsol uygulamaları yazabilmek.

1. Önce konuyu anla

Koleksiyonlar veriyi bir arada tutar; LINQ bu veriyi filtrelemek, dönüştürmek ve sıralamak için okunabilir bir ifade modeli sunar. `null` ise beklenen nesnenin hiç olmadığını anlatabilir ve kontrol edilmelidir.

GÜNLÜK HAYATTAN BENZETME

Sınıf listesindeki “70 üstlerini bul, isimlerini al, nota göre sırala” cümlesi LINQ düşüncesine benzer.

2. Kaynak programda ne öğrenilecek?

- Dizi, List<T>, Dictionary<TKey, TValue>
- LINQ'e giriş: Where, Select, OrderBy, Count,
- null kavramı ve null kontrolü

3. Uygulama / ödev

- ☐ 10 öğrenciden oluşan bir liste tanımlayınız (ad, sınıf, not)
- ☐ LINQ ile: notu 70 üstü olanlar, sınıfa göre ortalama, en yüksek
- ☐ Cuma değerlendirmesi: kodunuzu mentora satır satır anlatınız

4. Örnek üzerinden yavaşça ilerle

Gün 9 • örnek

```
var basarililar = students.Where(x => x.Not > 70);
var isimler = basarililar.Select(x => x.Ad);
var enYukse = students
    .OrderByDescending(x => x.Not)
    .FirstOrDefault();
```

5. Bu kodun fikri nedir?

- Where filtreler.
- Select projeksiyon/dönüşüm yapar.
- OrderByDescending sıralar.
- FirstOrDefault uygun sonuç yoksa varsayılan değer döndürebilir; null ihtimalini düşün.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Koleksiyonlar ve LINQ ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

LINQ alıştırmaları tamam; her sorgunun ne yaptığını açıklayabiliyorsunuz.

6. Kavrama soruları

- 1 decimal ile double arasındaki fark nedir? Para için hangisi kullanılır, neden?
- 2 List<T> ile dizi arasındaki fark nedir?
- 3 LINQ olmasaydı "notu 70 üstü öğrenciler" sorgusunu nasıl yazardınız? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 9 FAZ 1 · C# VE .NET Hafta 4 | Nesne Yönelimli Programlama (OOP) Haftanın hedefi: Sınıf, nesne, arayüz (interface) kavramlarını öğrenmek ve StajDesk'in ilk sürümünü konsolda yazmak.

Sınıflar ve nesneler

BUGÜNÜN HEDEFİ

Sınıf, nesne, interface kavramlarını öğrenmek ve StajDesk'in ilk sürümünü konsolda yazmak.

1. Önce konuyu anla

OOP, StajDesk'teki gerçek kavramları yazılıma modellemek için kullanılır. Ticket bir sınıf olabilir; oluşturulan belirli ticket ise nesnedir. Constructor başlangıç kurallarını uygular, encapsulation geçerli state'i korur.

GÜNLÜK HAYATTAN BENZETME

"Araba" model/şablondur; garajındaki belirli araba nesnedir.

2. Kaynak programda ne öğrenilecek?

- Sınıf ve nesne, özellik (property), constructor
- Erişim belirleyiciler (public, private) ve
- Değer tipi ve referans tipi farkı

3. Uygulama / ödev

- ☐ Ticket sınıfı oluşturunuz: Id, Title, Description, Status,
- ☐ Başlığın boş olamayacağını constructor içinde kontrol ediniz

4. Örnek üzerinden yavaşça ilerle

Gün 10 • örnek

```
public class Ticket
{
    public int Id { get; }
    public string Title { get; }

    public Ticket(int id, string title)
    {
        if (string.IsNullOrEmpty(title))
            throw new ArgumentException("Başlık boş olamaz.");
        Id = id;
        Title = title;
    }
}
```

5. Bu kodun fikri nedir?

- Property nesnenin durumunu temsil eder.
- private set gibi sınırlar kontrolsüz değişikliği azaltır.
- Value/reference type ayırımında kopyalama davranışını anlamak önemlidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Sınıflar ve nesneler ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Ticket sınıfı ve basit testleri çalışıyor.

Kalıtım, interface, enum, hata yönetimi

BUGÜNÜN HEDEFİ

Sınıf, nesne, interface kavramlarını öğrenmek ve StajDesk'in ilk sürümünü konsolda yazmak.

1. Önce konuyu anla

Interface bir sınıfın hangi işlemleri sunduğunu tarif eden sözleşmedir. Enum anlamlı sabit seçenekler verir. Exception ise başarısız durumların temsilidir. Repository interface'i sayesinde depolama yöntemi değişebilir.

GÜNLÜK HAYATTAN BENZETME

Bir iş sözleşmesinde görevler bellidir; çalışan kişinin görevi nasıl yaptığı farklı olabilir.

2. Kaynak programda ne öğrenilecek?

- Kalıtım (inheritance) ve polimorfizm
- Interface nedir, neden kullanılır ("sözleşme")
- enum ve try/catch/finally ile hata yönetimi

3. Uygulama / ödev

- ☐ TicketStatus enum'u tanımlayınız: Open, InProgress,
- ☐ ITicketRepository interface'i ve bunu uygulayan
- ☐ Olmayan bir Id istendiğinde anlamlı bir hata fırlatınız

4. Örnek üzerinden yavaşça ilerle

Gün 11 • örnek

```
public enum TicketStatus
{
    Open, InProgress, Resolved, Closed
}

public interface ITicketRepository
{
    Ticket Add(Ticket ticket);
    List<Ticket> GetAll();
    Ticket? GetById(int id);
}
```

5. Bu kodun fikri nedir?

- Ticket? null dönebileceğini açıkça ifade eder.
- Repository ile veri erişimi uygulamanın başka katmanlarından ayrılabilir.
- try/catch yalnızca anlamlı biçimde ele alınabilecek hatalarda kullanılmalıdır.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Kalıtım, interface, enum, hata yönetimi ne işe yarıyor?" sorusuna kendi cümlemle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Repository katmanı konsolda çalışıyor.

async/await ve Konsol Mini Talep Yöneticisi

BUGÜNÜN HEDEFİ

Sınıf, nesne, interface kavramlarını öğrenmek ve StajDesk'in ilk sürümünü konsolda yazmak.

1. Önce konuyu anla

async/await özellikle veritabanı veya ağ gibi bekleme içeren I/O işlemlerinde önemlidir. Task tamamlanacak işi temsil eder; await sonucu beklerken metodu uygun şekilde askıya alır.

GÜNLÜK HAYATTAN BENZETME

Garsonun siparişi beklerken boş boş durmak yerine başka masaya bakması gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Senkron ve asenkron çalışma; Task, async,
- Konsol menüsü tasarımı

3. Uygulama / ödev

- ☐ Menülü konsol uygulaması: talep ekle, listele, durum güncelle,
- ☐ Repository metotlarını async hâle getiriniz
- ☐ Cuma: mentor kod incelemesi; aldığınız geri bildirimleri

4. Örnek üzerinden yavaşça ilerle

Gün 12 • örnek

```
public async Task<Ticket?> GetByIdAsync(int id)
{
    return await _db.Tickets
        .FirstOrDefaultAsync(t => t.Id == id);
}
```

5. Bu kodun fikri nedir?

- async otomatik olarak her programı hızlandırmaz.
- await bloklayıcı `\.Result\`\.Wait()\`` zincirlerine göre web uygulamalarında daha uygun asenkron akış sağlar.
- Gerçek I/O için kütüphanenin async API'sini kullan.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "async/await ve Konsol Mini Talep Yöneticisi ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

StajDesk v0 (konsol) depoda.

6. Kavrama soruları

- 1** Interface kullanmanın bize ne faydası var? ITicketRepository örneği üzerinden anlatınız.
- 2** Kapsülleme neden önemlidir?
- 3** await kullanmasaydık web sunucusunda ne gibi bir sorun olurdu? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 10 FAZ 1 · C# VE .NET Hafta 5 | ASP.NET Core Web API Haftanın hedefi: Konsol uygulamasını, HTTP üzerinden erişilebilen bir REST API'ye dönüştürmek.

Web API'ye giriş

BUGÜNÜN HEDEFİ

Konsol uygulamasını HTTP üzerinden erişilebilen bir REST API'ye dönüştürmek.

1. Önce konuyu anla

Web API, başka istemcilerin HTTP üzerinden erişebileceği bir uygulama yüzüdür. ASP.NET Core'da istek middleware pipeline'dan geçer ve route edilen endpoint/controller tarafından işlenir. Swagger/OpenAPI keşif ve testte yardımcıdır.

GÜNLÜK HAYATTAN BENZETME

Danışma bankosu gibi düşün: kullanıcı talebini söyler, sistem doğru bölüme yönlendirir.

2. Kaynak programda ne öğrenilecek?

- API nedir; REST prensipleri (kaynak, URL, metot)
- dotnet new webapi; Program.cs, middleware
- Swagger/OpenAPI ile API'yi keşfetmek

3. Uygulama / ödev

- ☐ StajDesk.Api projesini oluşturunuz
- ☐ GET /api/health endpoint'i yazınız (sunucu saati ve "OK")
- ☐ Swagger arayüzünden deneyiniz

4. Örnek üzerinden yavaşça ilerle

Gün 13 • örnek

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/health", () =>
    Results.Ok(new { status = "OK", time = DateTime.UtcNow }));

app.Run();
```

5. Bu kodun fikri nedir?

- MapGet belirli bir GET endpoint'i oluşturur.
- 200 OK başarıyı ifade eder.
- OpenAPI API'nin sözleşmesini makinece anlaşılır biçimde tarif edebilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Web API'ye giriş ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

API ayağa kalkıyor, Swagger açılıyor.

Controller, routing ve Dependency Injection

BUGÜNÜN HEDEFİ

Konsol uygulamasını HTTP üzerinden erişilebilen bir REST API'ye dönüştürmek.

1. Önce konuyu anla

Controller HTTP isteklerini karşılayan giriş katmanıdır. Routing isteği doğru action'a götürür; model binding route/query/body verisini C# parametrelerine bağlar. DI controller'ın bağımlılıklarını dışarıdan sağlar.

GÜNLÜK HAYATTAN BENZETME

Otelde resepsiyon her hizmeti kendi yapmaz; doğru ekipten hizmet ister. DI buna benzer.

2. Kaynak programda ne öğrenilecek?

- Controller, route, model binding,
- Dependency Injection nedir; Singleton,
- Doğru durum kodlarını dönmek (200, 201, 204,

3. Uygulama / ödev

- ☐ TicketsController yazınız: GET (liste), GET /{id}, POST, PUT,
- ☐ Geçen haftaki InMemoryTicketRepository'yi DI ile kaydedip

4. Örnek üzerinden yavaşça ilerle

Gün 14 • örnek

```
builder.Services.AddScoped<ITicketRepository, InMemoryTicketRepository>();

[ApiController]
[Route("api/[controller]")]
public class TicketsController : ControllerBase
{
    public TicketsController(ITicketRepository repo) { }
}
```

5. Bu kodun fikri nedir?

- Scoped tipik web isteği scope'uyla ilişkilidir.
- NotFound 404, Ok 200, Created 201 ve NoContent 204 gibi sonuçlar API sözleşmesini anlatır.
- DI container dependency oluşturup inject etmeye yardım eder.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Controller, routing ve Dependency Injection ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

CRUD uçları Swagger'dan çalışıyor.

Katmanlı mimari ve DTO

BUGÜNÜN HEDEFİ

Konsol uygulamasını HTTP üzerinden erişilebilen bir REST API'ye dönüştürmek.

1. Önce konuyu anla

Katmanlı mimari sorumlulukları ayırır: Controller HTTP ile, Service iş kurallarıyla, Repository veri erişimiyle ilgilenir. DTO dışarıdaki API sözleşmesini entity modelinden ayırmaya yardım eder.

GÜNLÜK HAYATTAN BENZETME

Bir şirkette resepsiyon, uzman ve depo ekiplerinin her işin altına girmemesi gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Controller → Service → Repository katmanları;
- DTO (Data Transfer Object) nedir; neden

3. Uygulama / ödev

- ☐ TicketService oluşturup iş kurallarını buraya taşıyınız
- ☐ CreateTicketRequest ve TicketResponse DTO'larını
- ☐ Postman koleksiyonu hazırlayıp depoya ekleyiniz

4. Örnek üzerinden yavaşça ilerle

Gün 15 • örnek

```
public record CreateTicketRequest(
    string Title,
    string Description,
    int CategoryId);

public record TicketResponse(
    int Id,
    string Title,
    string Status);
```

5. Bu kodun fikri nedir?

- DTO istemcinin görmesi gereken alanları sınırlar.
- Service iş kuralları için merkezi bir yer sağlar.
- Controller'ın DB sorguları ve business rule ile dolmaması bakım kolaylığı sağlar.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Katmanlı mimari ve DTO ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Katmanlı API + Postman koleksiyonu depoda.

6. Kavrama soruları

- 1 Uygulamayı yeniden başlattığınızda talepler neden kayboluyor?
- 2 Dependency Injection olmasaydı TicketsController nasıl görünürdü?
- 3 POST başarılı olduğunda neden 200 yerine 201 döneriz? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 11 FAZ 2 · POSTGRESQL Hafta 6 | İlişkisel Veritabanı ve SQL Haftanın hedefi: Verinin kalıcı olarak nasıl saklandığını öğrenmek, SQL yazabilmek ve StajDesk veri modelini tasarlamak.

Veritabanı nedir, neden PostgreSQL?

BUGÜNÜN HEDEFİ

Verinin kalıcı saklanması öğrenmek, SQL yazabilmek ve StajDesk veri modelini tasarlamak.

1. Önce konuyu anla

İlişkisel veritabanında tablo satırları kayıt, sütunlar özellikleri temsil eder. Primary key satırı benzersiz tanımlamak için kullanılır. PostgreSQL SQL, transaction ve ilişkisel veri yönetimi için güçlü bir seçenektir.

GÜNLÜK HAYATTAN BENZETME

Excel'e benzeyen görünüm düşün ama arkasında ilişkiler, kısıtlar, transaction ve sorgu motoru vardır.

2. Kaynak programda ne öğrenilecek?

- İlişkisel model: tablo, satır, sütun, birincil anahtar
- Neden PostgreSQL: açık kaynak, güvenilir (ACID),
- Temel komutlar: CREATE TABLE, INSERT,

3. Uygulama / ödev

- ☐ Docker'daki PostgreSQL'e psql ile bağlanınız
- ☐ categories tablosunu elle oluşturup 5 kayıt ekleyiniz

4. Örnek üzerinden yavaşça ilerle

Gün 16 • örnek

```
CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);

INSERT INTO categories(name)
VALUES ('Teknik Destek'), ('Yazılım');

SELECT * FROM categories;
```

5. Bu kodun fikri nedir?

- PRIMARY KEY benzersizlik ve satır kimliği sağlar.
- NOT NULL boş değeri engeller.
- INSERT ekler, SELECT okur.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Veritabanı nedir, neden PostgreSQL? ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

İlk tablo ve kayıtlar hazır.

SQL sorguları ve ilişkiler

BUGÜNÜN HEDEFİ

Verinin kalıcı saklanması öğrenmek, SQL yazabilmek ve StajDesk veri modelini tasarlamak.

1. Önce konuyu anla

SQL veriyi filtrelemeyi, sıralamayı, güncellemeyi ve tabloları ilişkileri üzerinden birleştirmeyi sağlar. JOIN ilişkili satırları birleştirir; GROUP BY özetleme için kullanılır.

GÜNLÜK HAYATTAN BENZETME

Kütüphanede kitap tablosu ile ödünç tablosundan “kim hangi kitabı aldı?” sorusunu çıkarmak JOIN gibidir.

2. Kaynak programda ne öğrenilecek?

- WHERE, ORDER BY, UPDATE, DELETE
- Yabancı anahtar (foreign key), JOIN, GROUP BY,

3. Uygulama / ödev

- ☐ users, tickets, comments tablolarını ilişkileriyle oluşturunuz;
- ☐ Mentorun vereceği 10 soruya SQL ile cevap yazınız (ör.
- ☐ Sorguları /sql/week06.sql dosyasına kaydediniz

4. Örnek üzerinden yavaşça ilerle

Gün 17 • örnek

```
SELECT c.name, COUNT(t.id) AS open_count
FROM categories c
LEFT JOIN tickets t
  ON t.category_id = c.id
  AND t.status = 'Open'
GROUP BY c.id, c.name;
```

5. Bu kodun fikri nedir?

- UPDATE/DELETE'te WHERE koşulunu dikkatle yaz.
- LEFT JOIN soldaki tablonun eşleşmeyen satırlarını da korur.
- COUNT + GROUP BY özet raporların temel yapı taşlarındandır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “SQL sorguları ve ilişkiler ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

10 SQL sorgusu depoda.

Veri modelleme

BUGÜNÜN HEDEFİ

Verinin kalıcı saklanması öğrenmek, SQL yazabilmek ve StajDesk veri modelini tasarlamak.

1. Önce konuyu anla

ER diyagramı tablolar ve ilişkileri görsel olarak tasarlamaya yardım eder. 1-N ilişkide bir User birçok Ticket'a sahip olabilir. Index belirli sorguları hızlandırabilir ancak depolama ve yazma maliyeti getirir.

GÜNLÜK HAYATTAN BENZETME

Kitabın arkasındaki alfabetik indeks aramayı hızlandırır; fakat indeksi oluşturmak ve güncel tutmak da maliyetlidir.

2. Kaynak programda ne öğrenilecek?

- ER (varlık-iliski) diyagramı, 1-N ilişkiler
- Normalizasyona giriş; index nedir, neyi

3. Uygulama / ödev

- ☐ StajDesk ER diyagramını çizin (draw.io veya kâğıt): User,
- ☐ Hangi sütunlara index koyacağınızı gerekçesiyle yazınız
- ☐ Cuma: diyagramı mentorla gözden geçirin

4. Örnek üzerinden yavaşça ilerle

Gün 18 • örnek

```
CREATE INDEX ix_tickets_status
ON tickets(status);

CREATE INDEX ix_tickets_category_id
ON tickets(category_id);
```

5. Bu kodun fikri nedir?

- Index seçimi gerçek sorgu desenine göre yapılır.
- Her sütuna index koymak iyi bir genel kural değildir.
- Normalization tekrarları azaltmaya ve veri bütünlüğünü korumaya yardım eder.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Veri modelleme ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Onaylanmış ER diyagramı depoda (/docs).

6. Kavrama soruları

- 1** Talepleri bir metin dosyasında değil de veritabanında tutmamızın 3 nedeni nedir?
- 2** JOIN ne işe yarar? Kendi tablolarınızdan bir örnek veriniz.
- 3** Her sütuna index koymak neden iyi bir fikir değildir? Lise Stajyer Programı | StajDesk Projesi
info@devoplus.com.tr | www.devoplus.com.tr Sayfa 12 FAZ 2 · POSTGRESQL Hafta 7 | Entity
Framework Core ile API'yi Veritabanına Bağlamak Haftanın hedefi: ORM kavramını öğrenmek
ve API'nin verisini PostgreSQL'de kalıcı hâle getirmek.

ORM ve EF Core

BUGÜNÜN HEDEFİ

ORM kavramını öğrenmek ve API verisini PostgreSQL'de kalıcı hâle getirmek.

1. Önce konuyu anla

ORM, nesne dünyası ile ilişkisel veritabanı dünyası arasında bir eşleme katmanı oluşturur. EF Core .NET'in ORM çözümüdür; Npgsql PostgreSQL erişim sağlayıcısıdır; DbContext çalışma oturumunu yönetir.

GÜNLÜK HAYATTAN BENZETME

Türkçe konuşan yazılımcı ile SQL konuşan veritabanı arasında tercüman gibi düşün.

2. Kaynak programda ne öğrenilecek?

- ORM nedir; SQL yazmak ile ORM kullanmak
- EF Core, Npgsql sağlayıcısı, DbContext, entity
- Bağlantı cümlesi (connection string) ve

3. Uygulama / ödev

- ☐ Gerekli NuGet paketlerini ekleyiniz
- ☐ AppDbContext ve entity'leri (User, Category, Ticket,

4. Örnek üzerinden yavaşça ilerle

Gün 19 • örnek

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Ticket> Tickets => Set<Ticket>();
}
```

5. Bu kodun fikri nedir?

- DbSet entity kümesini temsil eder.
- Connection string DB sunucusu ve kimlik bilgileri gibi bağlantı ayarlarını taşır.
- ORM SQL bilgisini ortadan kaldırmaz; sorgunun veritabanında ne yapacağını anlamalısın.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "ORM ve EF Core ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Proje derleniyor, DbContext hazır.

Migration ve ilişkiler

BUGÜNÜN HEDEFİ

ORM kavramını öğrenmek ve API verisini PostgreSQL'de kalıcı hâle getirmek.

1. Önce konuyu anla

Migration, veritabanı şemasındaki değişiklikleri kod tarihçesiyle takip etmeye yarar. Navigation property ilişkili entity'lere nesne üzerinden erişimi kolaylaştırır; Fluent API ilişki ve kısıtları yapılandırabilir.

GÜNLÜK HAYATTAN BENZETME

Bir binanın her tadilatını adım adım kayıt altına almak gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Migration nedir, neden elle tablo oluşturmayız?
- Navigation property, Fluent API ile yapılandırma

3. Uygulama / ödev

- ☐ İlk migration'ı oluşturup veritabanına uygulayınız
- ☐ Tabloların psql ile oluştuğunu doğrulayınız
- ☐ Varsayılan kategorileri seed ediniz

4. Örnek üzerinden yavaşça ilerle

Gün 20 • örnek

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

5. Bu kodun fikri nedir?

- Migration ekip üyelerinin aynı şemaya ulaşmasına yardım eder.
- Production'da migration uygulama yöntemi ayrıca planlanmalıdır.
- Seed varsayılan referans verilerini oluşturabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Migration ve ilişkiler ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Migration'lar depoda, tablolar oluştu.

Repository'yi EF Core'a taşımak

BUGÜNÜN HEDEFİ

ORM kavramını öğrenmek ve API verisini PostgreSQL'de kalıcı hâle getirmek.

1. Önce konuyu anla

Gerçek veritabanında asenkron sorgular, yalnızca okumada tracking ihtiyacı yoksa AsNoTracking, ilişkili veri gerektiğinde Include ve büyük listeleri parçalara ayırmak için pagination önem kazanır.

GÜNLÜK HAYATTAN BENZETME

1 milyon kitabı bir seferde taşımak yerine raf raf almak gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Async sorgular, Include, AsNoTracking
- Sayfalama (Skip/Take) ve filtreleme

3. Uygulama / ödev

- ☐ EfTicketRepository yazıp DI kaydını değiştiriniz (interface)
- ☐ GET /api/tickets?status=Open&categoryId=2&page=1&p
- ☐ Cuma: API'yi yeniden başlatıp verinin kaldığını mentora

4. Örnek üzerinden yavaşça ilerle

Gün 21 • örnek

```
var items = await query
    .AsNoTracking()
    .OrderByDescending(t => t.Id)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
```

5. Bu kodun fikri nedir?

- Skip/Take klasik offset pagination örneğidir.
- AsNoTracking read-only sorgularda tracking maliyetini azaltabilir.
- Çok büyük veri setlerinde keyset/cursor pagination gibi seçenekler de düşünülebilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Repository'yi EF Core'a taşımak ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

API artık kalıcı veriyle çalışıyor.

6. Kavrama soruları

- 1** Repository'yi deęiřtirdięimizde controller'ın deęiřmemesi neyi kanıtlıyor?
- 2** Migration olmadan ekte 5 kiři aynı veritabanı řemasını nasıl senkron tutardı?
- 3** Sayfalama yapmasaydık 1 milyon talep olduęunda ne olurdu? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 13 FAZ 3 · GÜVENLİK Hafta 8 | Doğrulama, Hata Yönetimi ve Kimlik Doğrulama Haftanın hedefi: API'yi hatalı girdiye ve yetkisiz erişime karşı korumak; kayıt, giriş ve rol yönetimini eklemek.

Doğrulama ve hata yönetimi

BUGÜNÜN HEDEFİ

API'yi hatalı girdiye ve yetkisiz erişime karşı korumak; kayıt, giriş ve rol yönetimini eklemek.

1. Önce konuyu anla

Validation dışarıdan gelen verinin sistem kurallarına uygunluğunu kontrol eder. Frontend doğrulaması kullanıcı deneyimi için faydalıdır ama güvenlik sınırı değildir; backend kendi doğrulamasını yapmalıdır.

GÜNLÜK HAYATTAN BENZETME

Formu dolduran kişinin eksik evrağını gişede tekrar kontrol etmek gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Neden doğrulama her zaman sunucu tarafında
- DataAnnotations veya FluentValidation ile
- Global hata yakalama middleware'i ve standart

3. Uygulama / ödev

- ☐ Tüm request DTO'larına doğrulama kuralları ekleyiniz
- ☐ Tüm uçların aynı yanıt formatını dönmesini sağlayınız

4. Örnek üzerinden yavaşça ilerle

Gün 22 • örnek

```
if (string.IsNullOrEmpty(request.Title))
    return Results.BadRequest(new
    {
        message = "Başlık boş olamaz."
    });
```

5. Bu kodun fikri nedir?

- Hata yanıtında kullanıcıya gerekli bilgiyi ver; stack trace gibi iç detayları açma.
- Standart hata zarfı istemcinin işleri daha kolay yönetmesine yardım eder.
- Validation ile exception aynı şey değildir; beklenen geçersiz girdi normal akışın bir parçası olabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Doğrulama ve hata yönetimi ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Hatalı istekler anlaşılır 400 yanıtı alıyor.

Kayıt, giriş ve JWT

BUGÜNÜN HEDEFİ

API'yi hatalı girdiye ve yetkisiz erişime karşı korumak; kayıt, giriş ve rol yönetimini eklemek.

1. Önce konuyu anla

Authentication “kimsin?”, authorization “ne yapabilirsin?” sorusudur. Parolalar düz metin tutulmamalı; uygun password hashing yöntemi kullanılmalıdır. JWT'nin payload'ı normalde okunabilir; imza bütünlük ve doğrulama içindir.

GÜNLÜK HAYATTAN BENZETME

Kimlik kartı üzerindeki bilgilerin okunabilir olması ile kartın sahte olup olmadığının kontrol edilmesi farklı şeylerdir.

2. Kaynak programda ne öğrenilecek?

- Kimlik doğrulama (authentication) ile
- Şifreler neden düz metin saklanmaz; hash ve
- JWT'nin yapısı: header, payload, signature

3. Uygulama / ödev

- ☐ POST /api/auth/register ve POST /api/auth/login
- ☐ Şifreleri hash'leyerek saklayınız; login başarılıysa JWT
- ☐ Gizli anahtarları koda değil user-secrets / .env içine

4. Örnek üzerinden yavaşça ilerle

Gün 23 • örnek

```
POST /api/auth/login
Authorization: Bearer <access-token>

JWT:
header.payload.signature
```

5. Bu kodun fikri nedir?

- JWT payload'ına parola veya gizli anahtar koyma.
- Hashing ile encryption aynı problem değildir; parola doğrulama için geri döndürülmesi gerekmeyen hash yaklaşımı kullanılır.
- Secret key kaynak koda commit edilmemelidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Kayıt, giriş ve JWT ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Kayıt ve giriş çalışıyor; depoda hiçbir gizli anahtar yok.

Roller ve yetkilendirme

BUGÜNÜN HEDEFİ

API'yi hatalı girdiye ve yetkisiz erişime karşı korumak; kayıt, giriş ve rol yönetimini eklemek.

1. Önce konuyu anla

Yetkilendirme, sadece “token var mı?” kontrolü değildir. Kullanıcının rolü ve erişmeye çalıştığı kaynağın sahibi olup olmadığı da business rule olarak kontrol edilebilir.

GÜNLÜK HAYATTAN BENZETME

Okula girmek için kartın olması, öğretmenler odasına girebileceğin anlamına gelmez.

2. Kaynak programda ne öğrenilecek?

- [Authorize] ve rol bazlı yetki: User, Agent
- Kullanıcı yalnızca kendi taleplerini görmeli;

3. Uygulama / ödev

- ☐ Yetki kurallarını uygulayınız ve yorum ekleme ucunu (POST)
- ☐ Postman'da token ile istek atmayı koleksiyona ekleyiniz
- ☐ Cuma: başka bir kullanıcının talebine erişmeyi deneyip

4. Örnek üzerinden yavaşça ilerle

Gün 24 • örnek

```
[Authorize]
[HttpGet("{id:int}")]
public async Task<IActionResult> Get(int id)
{
    var ticket = await _service.GetAsync(id);
    if (ticket is null) return NotFound();

    if (!User.IsInRole("Agent"))
        return Forbid();

    return Ok(ticket);
}
```

5. Bu kodun fikri nedir?

- 401 kimlik doğrulama eksik/geçersiz durumudur.
- 403 kimliği doğrulanmış ama yetkisi olmayan kullanıcı içindir.
- Kullanıcı yalnızca kendi ticket'larını görebilecekse sahiplik kontrolü backend'de yapılır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Roller ve yetkilendirme ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

6. Kavrama soruları

- 1 JWT'nin içeriği okunabiliyorsa neden içine şifre veya gizli bilgi koymayız?
- 2 Frontend'de doğrulama yapıyorsak backend'de neden tekrar yapıyoruz?
- 3 401 ile 403 arasındaki fark nedir? Lise Stajyer Programı | StajDesk Projesi
info@devoplus.com.tr | www.devoplus.com.tr Sayfa 14 FAZ 4 · VUE.JS Hafta 9 | Frontend Temelleri ve Vue.js'e Giriş Haftanın hedefi: Tarayıcı tarafında neler olduğunu anlamak ve Vue'nun neden kullanıldığını deneyimleyerek görmek.

HTML/CSS/JavaScript tekrarı

BUGÜNÜN HEDEFİ

Tarayıcı tarafında neler olduğunu anlamak ve Vue'nun neden kullanıldığını deneyimlemek.

1. Önce konuyu anla

Vanilla JavaScript ile DOM'u doğrudan değiştirebilirsin. API'den veri almak için fetch kullanılabilir ve Promise tabanlı asenkron akış yönetilir. Vue, büyüyen UI'da state ve component ilişkisini daha sistematik hale getirir.

GÜNLÜK HAYATTAN BENZETME

Bir tahtadaki her yazıyı elle silip yeniden yazmak yerine değişiklikleri yöneten bir sistem kullanmak gibi düşün.

2. Kaynak programda ne öğrenilecek?

- DOM, olaylar (event), fetch, Promise
- Neden Vue? Bileşen yapısı, reaktivite, öğrenme

3. Uygulama / ödev

- ☐ Saf JavaScript ile API'den talepleri çekip listeleyen tek bir
- ☐ Aynı sayfaya filtre eklemeye çalışınız ve ne kadar zorlaştığını

4. Örnek üzerinden yavaşça ilerle

Gün 25 • örnek

```
async function loadTickets() {
  const response = await fetch('/api/tickets');
  const tickets = await response.json();
  // DOM güncelle
}
```

5. Bu kodun fikri nedir?

- DOM HTML'in programatik temsilidir.
- Event kullanıcı etkileşimlerini yakalar.
- Vue JavaScript'in yerine geçmez; JavaScript üzerine UI geliştirme modeli sunar.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "HTML/CSS/JavaScript tekrarı ne işe yapıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Vanilla JS sayfası çalışıyor; zorluklar not edildi.

Vue 3 temelleri

BUGÜNÜN HEDEFİ

Tarayıcı tarafında neler olduğunu anlamak ve Vue'nun neden kullanıldığını deneyimlemek.

1. Önce konuyu anla

Vue 3 Composition API'de `ref`, `reactive` ve `computed` sık kullanılan reactivity araçlarıdır. Direktifler template davranışını bildirir. Vite geliştirme ve build araç zincirinde kullanılır.

GÜNLÜK HAYATTAN BENZETME

Lego parçaları gibi düşün: component'ler ve reaktif state birleşerek ekranı oluşturur.

2. Kaynak programda ne öğrenecek?

- Vite ile proje oluşturma; .vue dosya yapısı
- Composition API: ref, reactive, computed
- Direktifler: v-if, v-for, v-model, @click

3. Uygulama / ödev

- ☐ stajdesk-web projesini oluşturunuz
- ☐ Savaş ve yapılacaklar listesi (to-do) bileşenleri yazınız

4. Örnek üzerinden yavaşça ilerle

Gün 26 • örnek

```
<script setup>
import { ref, computed } from 'vue'

const count = ref(0)
const total = computed(() => count.value + 1)
</script>

<template>
  <button @click="count++">{{ count }}</button>
</template>
```

5. Bu kodun fikri nedir?

- ref bir reaktif değer tutar.
- computed başka state'ten türeyen sonucu ifade eder.
- v-if koşul, v-for liste, v-model form bağlantısı, @click event bağlama örneğidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Vue 3 temelleri ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Vue projesi çalışıyor, iki alıştırmaya tamam.

Bileşenler

BUGÜNÜN HEDEFİ

Tarayıcı tarafında neler olduğunu anlamak ve Vue'nun neden kullanıldığını deneyimlemek.

1. Önce konuyu anla

Component'ler ekranı bağımsız sorumluluklara ayırır. Props parent'tan child'a veri taşır; emit child'ın parent'a olay bildirmesine yardım eder; slot component içinde özelleştirilebilir içerik alanı sağlar.

GÜNLÜK HAYATTAN BENZETME

Bir sipariş formunda bilgi aşağı doğru gelir; tamamlandı olayı yukarı bildirilir.

2. Kaynak programda ne öğrenilecek?

- Bileşen, props, emit, slot
- Klasör yapısı: views, components, services,

3. Uygulama / ödev

- ☐ TicketCard ve StatusBadge bileşenlerini yazınız
- ☐ Sahte (mock) veriyle talep listesi sayfası hazırlayınız
- ☐ Cuma: pazartesi yazdığınız vanilla JS ile Vue sürümünü

4. Örnek üzerinden yavaşça ilerle

Gün 27 • örnek

```
<script setup>
defineProps({ title: String })
const emit = defineEmits(['select'])
</script>

<template>
<article @click="emit('select')">
  {{ title }}
</article>
</template>
```

5. Bu kodun fikri nedir?

- Props doğrudan değiştirilmemelidir; child olay emit edebilir.
- Component'leri ayırmak tekrar kullanım ve bakım sağlar.
- views sayfa düzeyi, components küçük UI parçaları, services API işleri, stores ortak state için düşünülebilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Bileşenler ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Bileşen tabanlı liste sayfası.

6. Kavrama soruları

- 1** Reaktivite nedir? Vue olmadan aynı etkiyi nasıl elde ederdiniz?
- 2** props ile emit arasındaki veri akışını çizerek anlatınız.
- 3** Neden sayfayı tek bir büyük dosya yerine bileşenlere böleriz? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 15 FAZ 4 · VUE.JS Hafta 10 | Vue Router, Pinia ve API Bağlantısı Haftanın hedefi: Frontend'i gerçek API'ye bağlamak; sayfa yönlendirme ve oturum yönetimini kurmak.

Vue Router

BUGÜNÜN HEDEFİ

Frontend'i gerçek API'ye bağlamak; sayfa yönlendirme ve oturum yönetimini kurmak.

1. Önce konuyu anla

Router, SPA içinde URL ile gösterilecek view arasındaki ilişkiyi yönetir. Dinamik route `/tickets/:id` bir parametre taşır. Navigation guard, kullanıcıyı doğru sayfaya yönlendirmek için kullanılabilir.

GÜNLÜK HAYATTAN BENZETME

Bir binada hangi numaralı odanın nerede olduğunu gösteren yönlendirme tabelaları gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Sayfa yönlendirme, dinamik route
- Navigation guard ile sayfa koruma

3. Uygulama / ödev

- ☐ Sayfa iskeletlerini oluşturunuz: Giriş, Kayıt, Taleplerim, Talep

4. Örnek üzerinden yavaşça ilerle

Gün 28 • örnek

```
const routes = [
  { path: '/login', component: LoginView },
  { path: '/tickets/:id', component: TicketDetailView }
]
```

5. Bu kodun fikri nedir?

- Dinamik parametre route'dan okunabilir.
- Guard kullanıcı deneyiminde koruma sağlar.
- Asıl yetki kontrolü backend'de kalmalıdır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Vue Router ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Sayfalar arası gezinme çalışıyor.

API bağlantısı ve CORS

BUGÜNÜN HEDEFİ

Frontend'i gerçek API'ye bağlamak; sayfa yönlendirme ve oturum yönetimini kurmak.

1. Önce konuyu anla

Axios veya fetch ile API'ye istek atılır. CORS, tarayıcının farklı origin'lere yaptığı isteklerde uyguladığı güvenlik modelidir. Vite client'a taşınan `VITE\_` değişkenleri gizli sayılmamalıdır.

GÜNLÜK HAYATTAN BENZETME

Bir binanın güvenliği, başka bloktan gelen ziyaretçinin geçiş kuralını kontrol ediyor gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Axios ile istek atma; servis katmanı
- CORS nedir, neden tarayıcı engeller; API'de nasıl
- Ortam değişkenleri (.env, VITE_API_URL)

3. Uygulama / ödev

- ☐ Kayıt ve giriş formlarını API'ye bağlayınız
- ☐ Hata mesajlarını kullanıcıya resmî dille gösteriniz (ör. "E-posta

4. Örnek üzerinden yavaşça ilerle

Gün 29 • örnek

```
import axios from 'axios'

export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL
})
```

5. Bu kodun fikri nedir?

- CORS hatası browser politikasıyla ilişkilidir; Postman aynı browser kısıtlarına tabi değildir.
- API URL environment ile ayrıştırılabilir.
- VITE_ ile client bundle'a girebilen bir değer secret olmamalıdır.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "API bağlantısı ve CORS ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Frontend üzerinden kayıt ve giriş yapılabilir.

Pinia ile oturum yönetimi

BUGÜNÜN HEDEFİ

Frontend'i gerçek API'ye bağlamak; sayfa yönlendirme ve oturum yönetimini kurmak.

1. Önce konuyu anla

Pinia ortak state'i merkezi bir store'da tutmaya yardım eder. Auth store token, kullanıcı ve rol gibi istemci durumlarını tek yerde yönetebilir. Axios interceptor ortak header ve hata davranışları için kullanılabilir.

GÜNLÜK HAYATTAN BENZETME

Okuldaki ortak duyuru panosu gibi düşün: bütün sınıflar aynı bilgiyi farklı defterlerde tekrar tutmaz.

2. Kaynak programda ne öğrenilecek?

- State management nedir; Pinia store
- Axios interceptor ile her isteğe token eklemek;

3. Uygulama / ödev

- ☐ auth store'u yazınız (token, kullanıcı, rol)
- ☐ Giriş yapmamış kullanıcıyı korumalı sayfalardan Giriş
- ☐ Cuma: uçtan uca giriş akışını gösteriniz

4. Örnek üzerinden yavaşça ilerle

Gün 30 • örnek

```
export const useAuthStore = defineStore('auth', {
  state: () => ({
    token: null,
    user: null,
    role: null
  }),
  actions: {
    logout() {
      this.token = null
      this.user = null
      this.role = null
    }
  }
})
```

5. Bu kodun fikri nedir?

- Store tekrar eden auth durumunu azaltır.
- 401 geldiğinde oturum kapatma davranışı merkezi düşünülebilir.
- Frontend store güvenlik otoritesi değildir; backend token ve yetkiyi doğrular.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Pinia ile oturum yönetimi ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Oturum yönetimi tamam.

6. Kavrama soruları

- 1 CORS hatası neden Postman'da değil de tarayıcıda çıkar?
- 2 Token'ı her bileşende ayrı ayrı tutmak yerine neden store kullanıyoruz?
- 3 Frontend'deki route guard güvenlik için yeterli midir? Neden? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 16 FAZ 4 · VUE.JS Hafta 11 | Uygulama Ekranları Haftanın hedefi: StajDesk'in tüm kullanıcı ekranlarını tamamlamak; uygulamayı uçtan uca kullanılabilir hâle getirmek.

Talep listesi

BUGÜNÜN HEDEFİ

Tüm kullanıcı ekranlarını tamamlamak ve uygulamayı uçtan uca kullanılabilir hâle getirmek.

1. Önce konuyu anla

Kullanıcı veri beklerken ne olduğunu görmelidir. Loading, empty ve error durumlarını ayrı modellemek daha iyi UX sağlar. Filtre ve pagination ise hem kullanılabilirlik hem de veri yükü açısından önemlidir.

GÜNLÜK HAYATTAN BENZETME

Restoranda yemek hazırlanıyor, bugün yok veya sistemde sorun var gibi üç ayrı durum düşün.

2. Kaynak programda ne öğrenilecek?

- Liste ekranı tasarımı: filtre, sayfalama
- Yükleniyor, boş liste ve hata durumlarının

3. Uygulama / ödev

- ☐ Taleplerim sayfasını API'ye bağlayınız; durum ve kategori
- ☐ Üç durumun (yükleniyor/boş/hata) her birini ekranda

4. Örnek üzerinden yavaşça ilerle

Gün 31 • örnek

```
<p v-if="loading">Yükleniyor...</p>
<p v-else-if="error">Bir hata oluştu.</p>
<p v-else-if="tickets.length === 0">Henüz talep yok.</p>
<ul v-else>...</ul>
```

5. Bu kodun fikri nedir?

- Her ekran yalnızca “başarılı veri” durumundan ibaret değildir.
- Hata mesajları kullanıcıya uygun olmalıdır.
- Frontend pagination backend pagination ile eşleşmelidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Talep listesi ne işe yarıyor?” sorusuna kendi cümlemle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Talep listesi tamam.

Talep detayı ve yeni talep

BUGÜNÜN HEDEFİ

Tüm kullanıcı ekranlarını tamamlamak ve uygulamayı uçtan uca kullanılabilir hâle getirmek.

1. Önce konuyu anla

Form doğrulaması kullanıcıya erken geri bildirim verir. Rol bazlı arayüz, yanlış butonların görünmesini önleyebilir; fakat backend authorization'ın yerine geçmez.

GÜNLÜK HAYATTAN BENZETME

Bir formda eksik alanı daha gişeye gitmeden fark etmek gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Form doğrulama ve kullanıcı dostu hata
- Rol bazlı arayüz: Agent'ın görebildiği ek butonlar

3. Uygulama / ödev

- ☐ Talep detayı: açıklama, yorumlar, yorum ekleme
- ☐ Agent rolü için durum değiştirme
- ☐ Yeni talep formu (başlık, açıklama, kategori) ve doğrulama

4. Örnek üzerinden yavaşça ilerle

Gün 32 • örnek

```
if (!form.title.trim()) {
  errors.title = 'Başlık alanı zorunludur.'
}
if (form.description.length < 10) {
  errors.description = 'Açıklama çok kısa.'
}
```

5. Bu kodun fikri nedir?

- UI validation kullanıcı deneyimidir.
- Backend validation güvenlik sınırır.
- Agent'a özel buton sadece görünürlüğü düzenler; API ayrıca yetki kontrol eder.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Talep detayı ve yeni talep ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Talep oluşturma ve takip akışı tamam.

Yönetim ekranı ve özet paneli

BUGÜNÜN HEDEFİ

Tüm kullanıcı ekranlarını tamamlamak ve uygulamayı uçtan uca kullanılabilir hâle getirmek.

1. Önce konuyu anla

Dashboard, ham veriyi karar verilebilir özetlere çevirir. İyi dashboard gereksiz binlerce veri yerine kritik metrikleri gösterir ve API bunu uygun sorgularla üretir.

GÜNLÜK HAYATTAN BENZETME

Arabanın gösterge paneli tüm motoru göstermez; önemli bilgileri özetler.

2. Kaynak programda ne öğrenilecek?

- Basit bir özet paneli (dashboard) için API
- Responsive (mobil uyumlu) düzen

3. Uygulama / ödev

- ☐ GET /api/stats ucu: duruma göre talep sayıları
- ☐ Admin için kategori yönetimi ve özet paneli ekranı
- ☐ Cuma DEMO: kayıt → talep aç → Agent yanıtlasın → talep

4. Örnek üzerinden yavaşça ilerle

Gün 33 • örnek

```
GET /api/stats
```

```
{
  "open": 12,
  "inProgress": 7,
  "resolved": 18,
  "closed": 43
}
```

5. Bu kodun fikri nedir?

- Dashboard açıldığında kaç DB sorgusu çalıştığını düşün.
- Stats sorguları büyüdükçe indeks, aggregation ve cache ihtiyaçları gündeme gelebilir.
- Responsive tasarım farklı ekran genişliklerinde kullanılabilirlik sağlar.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Yönetim ekranı ve özet paneli ne işe yapıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

StajDesk uçtan uca çalışıyor (MVP).

6. Kavrama soruları

- 1 Uygulamanın řu anki hâlinde performans sorunu çıkabilecek yerler nereler?
- 2 Özet paneli her açıldığında veritabanında hangi sorgular çalışıyor?
- 3 Kullanıcı deneyimi açısından en çok neyi geliřtirmek isterdiniz? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 17 FAZ 5 · REDİS Hafta 12 | Redis ile Önbellekleme ve Hız Sınırlama Haftanın hedefi: Redis'in ne olduđunu, veritabanından farkını ve neden kullanıldığını ölçerek öğrenmek.

Redis nedir?

BUGÜNÜN HEDEFİ

Redis'in ne olduğunu, veritabanından farkını ve neden kullanıldığını ölçerek öğrenmek.

1. Önce konuyu anla

Redis bellek içi veri yapıları ve hızlı erişim için kullanılır. String, hash, list, set ve sorted set gibi veri yapıları farklı problemlere uyar. TTL bir anahtarın yaşam süresini sınırlayabilir.

GÜNLÜK HAYATTAN BENZETME

PostgreSQL arşiv odasıysa Redis masanın üzerindeki hızlı not defteridir.

2. Kaynak programda ne öğrenilecek?

- Bellek içi (in-memory) anahtar-değer deposu;
- Redis neden PostgreSQL'in yerini tutmaz,
- Veri tipleri: string, hash, list, set, sorted set; TTL

3. Uygulama / ödev

- ☐ compose dosyasına Redis ekleyiniz
- ☐ redis-cli ile SET, GET, EXPIRE, TTL, INCR, DEL komutlarını
- ☐ Her komutun ne yaptığını LOG.md'ye örneğiyle yazınız

4. Örnek üzerinden yavaşça ilerle

Gün 34 • örnek

```
SET app:status "OK"
GET app:status
SET login:fail:user 1 EX 300
INCR login:fail:user
TTL login:fail:user
DEL login:fail:user
```

5. Bu kodun fikri nedir?

- Redis cache ve sayaç gibi senaryolarda güçlüdür.
- PostgreSQL ile aynı amaçta düşünülmemelidir.
- TTL stale verinin ömrünü sınırlamada yardımcı olabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Redis nedir? ne işe yarıyor?" sorusuna kendi cümlemle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Redis çalışıyor, komut notları hazır.

Önbellekleme (cache-aside)

BUGÜNÜN HEDEFİ

Redis'in ne olduğunu, veritabanından farkını ve neden kullanıldığını ölçerek öğrenmek.

1. Önce konuyu anla

Cache-aside deseninde uygulama önce cache'e bakar. Hit varsa hızlı veri döner; miss varsa veritabanına gidilir ve sonuç cache'e yazılır. Veri değiştiğinde invalidation stale sonuç riskini azaltır.

GÜNLÜK HAYATTAN BENZETME

Sık baktığın ders sayfasının fotokopisini masada tutmak gibi düşün; kitap değişirse kopya güncellenmeli.

2. Kaynak programda ne öğrenilecek?

- Cache-aside deseni: önce önbelleğe bak, yoksa
- Önbellek geçersiz kılma (cache invalidation):
- StackExchange.Redis veya

3. Uygulama / ödev

- ☐ Kategori listesi ve /api/stats sonuçlarını 60 saniyeliğine
- ☐ Kategori eklendiğinde/değiştirildiğinde ilgili önbelleği siliniz
- ☐ Stopwatch ile önbellekli ve önbelleksiz süreleri ölçüp

4. Örnek üzerinden yavaşça ilerle

Gün 35 • örnek

```
var cached = await cache.GetStringAsync(key);

if (cached is not null)
    return Results.Content(cached, "application/json");

var result = await service.GetStatsAsync();
await cache.SetStringAsync(key, Serialize(result), options);
return Results.Ok(result);
```

5. Bu kodun fikri nedir?

- Cache source of truth değildir.
- TTL ve invalidation veri tazeliğiyle hız arasında denge kurar.
- Ölçüm yapmadan cache'in faydalı olduğunu varsayma.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Önbellekleme (cache-aside) ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Ölçümlerle kanıtlanmış önbellekleme.

Hız sınırlama (rate limiting)

BUGÜNÜN HEDEFİ

Redis'in ne olduğunu, veritabanından farkını ve neden kullanıldığını ölçerek öğrenmek.

1. Önce konuyu anla

Rate limiting belirli bir kaynak için istek sayısını sınırlar. Redis ortak sayaç sağlayabildiği için dağıtık uygulamalarda kullanılabilir. INCR + EXPIRE basit fixed-window sayaç desenidir.

GÜNLÜK HAYATTAN BENZETME

Turnikede kısa sürede sürekli kart okutmayı sınırlamak gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Brute-force saldırısı nedir?
- Redis INCR + EXPIRE ile sayaç tabanlı hız

3. Uygulama / ödev

- ☐ Aynı e-posta ile 5 dakikada en fazla 5 hatalı giriş denemesine
- ☐ Limit aşılmıca 429 dönünüz; frontend'de "Çok fazla deneme
- ☐ Cuma: Redis konteynerini durdurup uygulamanın davranışını

4. Örnek üzerinden yavaşça ilerle

Gün 36 • örnek

```
var count = await redis.StringIncrementAsync(key);

if (count == 1)
    await redis.KeyExpireAsync(key, TimeSpan.FromMinutes(5));

if (count > 5)
    return Results.StatusCode(429);
```

5. Bu kodun fikri nedir?

- 429 Too Many Requests'tir.
- Üretimde yarış koşulları ve daha gelişmiş algoritmalar ayrıca düşünülür.
- Redis kapanırsa rate limit davranışını bilinçli tasarlamak gerekir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Hız sınırlama (rate limiting) ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Giriş ucu brute-force'a karşı korumalı.

6. Kavrama soruları

- 1** Redis'i ana veritabanı olarak kullansaydık ne gibi riskler olurdu?
- 2** Önbellekteki veri eskidiğinde (stale) kullanıcı ne görür? Bu kabul edilebilir mi?
- 3** Redis kapanırsa uygulama tamamen durmalı mı, yoksa veritabanıyla devam mı etmeli? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 18 FAZ 6 · RABBITMQ Hafta 13 | RabbitMQ ile Asenkron Mesajlaşma Haftanın hedefi: Mesaj kuyruğu mantığını öğrenmek; bildirimleri API'den bağımsız çalışan bir arka plan servisine taşımak.

Mesaj kuyruğu nedir?

BUGÜNÜN HEDEFİ

Mesaj kuyruğu mantığını öğrenmek; bildirimleri API'den bağımsız çalışan worker'a taşımak.

1. Önce konuyu anla

Mesaj kuyruğu, üretici ile tüketiciyi doğrudan ve aynı anda çalışmak zorunda olmaktan kurtarabilir. Producer mesaj üretir, exchange yönlendirir, queue bekletir, consumer işler. Dayanıklılık için queue, mesaj ve acknowledgement ayarları birlikte değerlendirilmelidir.

GÜNLÜK HAYATTAN BENZETME

Kafedeki sipariş fişlerinin sıraya konması gibi düşün.

2. Kaynak programda ne öğrenilecek?

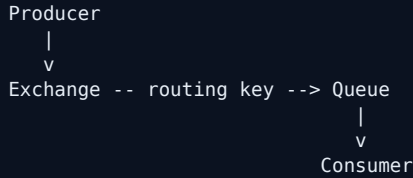
- Senkron ve asenkron iletişim; işleri birbirinden
- Producer, consumer, queue, exchange, routing
- Neden RabbitMQ: yük altında dayanıklılık, mesaj

3. Uygulama / ödev

- ☐ compose dosyasına rabbitmq:management ekleyiniz;
- ☐ Arayüzden elle kuyruk oluşturup mesaj yayınlayınız ve
- ☐ RabbitMQ resmî "Hello World" öğreticisini C# ile tamamlayınız

4. Örnek üzerinden yavaşça ilerle

Gün 37 • örnek



5. Bu kodun fikri nedir?

- Exchange mesajı uygun queue'lara yönlendirebilir.
- Durable queue tek başına “mesaj kesin kaybolmaz” demek değildir.
- Publisher confirms ve consumer acknowledgements farklı güvenilirlik noktalarını ele alır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Mesaj kuyruğu nedir? ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

İlk producer/consumer çalışıyor.

Olay yayınlama ve Worker servisi

BUGÜNÜN HEDEFİ

Mesaj kuyruğu mantığını öğrenmek; bildirimleri API'den bağımsız çalışan worker'a taşımak.

1. Önce konuyu anla

Event, sistemde gerçekleşen bir olayı ifade eder. Worker Service arka planda bu olayları tüketip bildirim veya başka yan işler yapabilir. Ack/nack mesajın işlenme durumunu broker'a bildirmeye yardım eder.

GÜNLÜK HAYATTAN BENZETME

Bir duyuru panosu düşün: API “talep oluştu” duyurusunu yayınlar, worker ilgilenir.

2. Kaynak programda ne öğrenilecek?

- Olay (event) tasarımı: TicketCreated,
- Worker Service (dotnet new worker),
- Kalıcı kuyruk (durable), mesaj onayı (ack/nack)

3. Uygulama / ödev

- ☐ API'de talep oluşturulduğunda ve durum değiştiğinde olayı
- ☐ StajDesk.Worker projesini oluşturup bu olayları dinleyiniz

4. Örnek üzerinden yavaşça ilerle

Gün 38 • örnek

```
public record TicketCreated(
    int TicketId,
    int UserId,
    DateTime CreatedAt);

await HandleAsync(message);

consumer.Ack(message);
```

5. Bu kodun fikri nedir?

- Ack zamanlaması iş başarıyla tamamlandıktan sonra tasarlanır.
- Nack/queue sonsuz döngüye dönüşmemeli; retry/dead-letter stratejileri düşünülmelidir.
- Worker'ın bağımsız olması API yanıt süresini iyileştirebilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Olay yayınlama ve Worker servisi ne işe yapıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

API olay yayınlıyor, Worker dinliyor.

Bildirimler

BUGÜNÜN HEDEFİ

Mesaj kuyruğu mantığını öğrenmek; bildirimleri API'den bağımsız çalışan worker'a taşımak.

1. Önce konuyu anla

Worker event'i kalıcı notification kaydına çevirebilir. Frontend polling ile belli aralıklarla API'yi kontrol ederek bildirimleri gösterebilir. Bu basitlik ile gecikme/istek sayısı arasında bir denge kurar.

GÜNLÜK HAYATTAN BENZETME

Posta kutusuna bırakılan mektubun postacı çalıştığında teslim alınması gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Worker'ın veritabanına yazması; bildirim modeli
- Frontend'de periyodik yenileme (polling)

3. Uygulama / ödev

- ☐ Worker gelen olaylardan notifications tablosuna kayıt
- ☐ GET /api/notifications ucu ve Vue'da bildirim zili
- ☐ Deney: Worker'ı durdurun, 3 talep açın, Worker'ı başlatın;

4. Örnek üzerinden yavaşça ilerle

Gün 39 • örnek

```
GET /api/notifications

[
  {
    "id": 42,
    "message": "Talebiniz güncellendi.",
    "isRead": false
  }
]
```

5. Bu kodun fikri nedir?

- Polling sıklığı arttıkça gecikme azalabilir ama istek sayısı artar.
- Worker kapalıyken mesajların korunması queue/message durability tasarımına bağlıdır.
- Notification DB'de tutulursa kullanıcı sonradan da görebilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Bildirimler ne işe yarıyor?" sorusuna kendi cümlemle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Asenkron bildirim sistemi çalışıyor.

6. Kavrama soruları

- 1** Bildirimi doğrudan API içinde göndermek yerine neden kuyruğa atıyoruz?
- 2** Worker kapalıyken gelen mesajlara ne oldu? Bu neden önemli?
- 3** Mesaj işlenirken hata olursa ack göndermezsek ne olur? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 19 FAZ 7 · BÜTÜNLEŞTİRME Hafta 14 | Tüm Sistemin Docker ile Çalıştırılması Haftanın hedefi: Tek bir komutla (docker compose up) tüm sistemin herhangi bir bilgisayarda ayağa kalkmasını sağlamak.

Backend Dockerfile'ları

BUGÜNÜN HEDEFİ

Tek komutla tüm sistemin farklı bir bilgisayarda ayağa kalkabilmesini sağlamak.

1. Önce konuyu anla

Multi-stage Docker build ile derleme ortamı ile çalışma ortamı ayrılır. SDK build için gerekir; final image yalnızca uygulamayı çalıştırmak için gereken runtime ve yayınlanan çıktıyı taşıyabilir.

GÜNLÜK HAYATTAN BENZETME

Büyük atölyede üretip müşteriye sadece bitmiş ürünü göndermek gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Çok aşamalı (multi-stage) build: SDK ile derle,
- .dockerignore; image boyutunu küçültmek

3. Uygulama / ödev

- ☐ API ve Worker için Dockerfile yazınız
- ☐ Image boyutlarını karşılaştırıp not ediniz (SDK vs runtime)

4. Örnek üzerinden yavaşça ilerle

Gün 40 • örnek

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "StajDesk.Api.dll"]
```

5. Bu kodun fikri nedir?

- Multi-stage final image'ın boyutunu ve saldırı yüzeyini azaltabilir.
- SDK ile runtime aynı sorumluluk değildir.
- `.dockerignore` gereksiz dosyaların build context'e taşınmasını azaltabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Backend Dockerfile'ları ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

API ve Worker konteynerde çalışıyor.

Frontend Dockerfile'ı

BUGÜNÜN HEDEFİ

Tek komutla tüm sistemin farklı bir bilgisayarda ayağa kalkabilmesini sağlamak.

1. Önce konuyu anla

Vue production build'i statik dosyalara dönüşebilir. Nginx bu dosyaları sunabilir ve reverse proxy ile `/api` isteklerini backend servisine iletebilir.

GÜNLÜK HAYATTAN BENZETME

AVM danışma masası gibi: tek girişten gelen istek doğru bölüme yönlendir.

2. Kaynak programda ne öğrenilecek?

- Vue projesini build etmek; statik dosyaları nginx
- nginx ile /api isteklerini backend'e

3. Uygulama / ödev

- ☐ Frontend Dockerfile'ı ve nginx.conf yazınız

4. Örnek üzerinden yavaşça ilerle

Gün 41 • örnek

```
FROM node:alpine AS build
WORKDIR /app
COPY package*.json .
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

5. Bu kodun fikri nedir?

- Node stage build içindir.
- Nginx production statik sunum için seçilebilir.
- Reverse proxy ayarı Docker Compose servis adını upstream olarak kullanabilir.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Frontend Dockerfile'ı ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Frontend konteynerde çalışıyor.

Tek compose dosyası

BUGÜNÜN HEDEFİ

Tek komutla tüm sistemin farklı bir bilgisayarda ayağa kalkabilmesini sağlamak.

1. Önce konuyu anla

Tek Compose dosyası tüm stack'i ayağa kaldırmayı kolaylaştırır. `depends\_on` başlama bağımlılığını ifade eder; healthcheck ise servis hakkında sağlık sinyali üretir. Environment değerleri paylaşım ve güvenlik açısından kontrollü yönetilmelidir.

GÜNLÜK HAYATTAN BENZETME

Orkestrada sahneye çıkmak ile enstrümanın gerçekten hazır olması aynı şey değildir.

2. Kaynak programda ne öğrenilecek?

- depends_on ve healthcheck; servislerin
- Ortam değişkenleri ve .env dosyası (.env)

3. Uygulama / ödev

- ☐ postgres, redis, rabbitmq, api, worker, web servislerini tek
- ☐ API açılışında migration ve seed otomatik çalışsın
- ☐ Cuma: depoyu temiz bir bilgisayara klonlayıp docker

4. Örnek üzerinden yavaşça ilerle

Gün 42 • örnek

```
services:
  postgres:
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U stajdesk"]
      interval: 5s
      timeout: 5s
      retries: 10

  api:
    depends_on:
      postgres:
        condition: service_healthy
```

5. Bu kodun fikri nedir?

- depends_on tek başına readiness garantisi değildir.
- Gerçek üretim migration yaklaşımı ayrıca planlanmalıdır.
- `.env` içine sır konulacaksa repo dışı tutulmalı; paylaşılabilir şablon `.env.example` olabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Tek compose dosyası ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Tek komutla ayağa kalkan sistem.

6. Kavrama soruları

- 1 Multi-stage build neden image boyutunu küçültür?
- 2 depends_on tek başına servisin hazır olduğunu garanti eder mi?
- 3 .env dosyasını neden depoya koymuyoruz? Lise Stajyer Programı | StajDesk Projesi
info@devoplus.com.tr | www.devoplus.com.tr Sayfa 20 FAZ 7 · BÜTÜNLEŞTİRME Hafta 15 |
Kalite: Test, Loglama ve Dokümantasyon Haftanın hedefi: Yazılan kodun doğruluğunu test etmek, sorunları izlenebilir kılmak ve projeyi belgelemek.

Birim testler

BUGÜNÜN HEDEFİ

Yazılan kodun doğruluğunu test etmek, sorunları izlenebilir kılmak ve projeyi belgelemek.

1. Önce konuyu anla

Birim test küçük bir davranışı izole doğrular. Arrange veriyi hazırlar, Act davranışı çalıştırır, Assert sonucu kontrol eder. Test sayısından çok doğru risklerin test edilmesi önemlidir.

GÜNLÜK HAYATTAN BENZETME

Bir makinenin parçalarını tek tek kalite kontrolünden geçirmek gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Birim test nedir, neden yazılır;
- xUnit ile test projesi

3. Uygulama / ödev

- ☐ TicketService için en az 8 birim test yazınız

4. Örnek üzerinden yavaşça ilerle

Gün 43 • örnek

```
[Fact]
public void Create_WithEmptyTitle_ShouldFail()
{
    // Arrange
    // Act
    // Assert
}
```

5. Bu kodun fikri nedir?

- İyi test adı beklenen davranışı söyler.
- Mutlu yol kadar geçersiz durumları da test et.
- dotnet test testleri otomatik çalıştırabilir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Birim testler ne işe yarıyor?" sorusuna kendi cümlemle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Testler yeşil (dotnet test).

Loglama ve güvenlik kontrolü

BUGÜNÜN HEDEFİ

Yazılan kodun doğruluğunu test etmek, sorunları izlenebilir kılmak ve projeyi belgelemek.

1. Önce konuyu anla

Loglar üretimde sorun araştırmayı kolaylaştırır. Information normal akış, Warning dikkat gerektiren durum, Error başarısızlık gibi düşünülebilir. Parola, token ve benzeri hassas sırlar loglanmamalıdır.

GÜNLÜK HAYATTAN BENZETME

Kara kutu gibi düşün: teşhis için gerekli veriyi kaydet, gizli anahtarı değil.

2. Kaynak programda ne öğrenilecek?

- ILogger, log seviyeleri (Information, Warning,
- Loga ne yazılır, ne yazılmaz (şifre, token asla)

3. Uygulama / ödev

- ☐ Önemli noktalara anlamlı loglar ekleyiniz
- ☐ Güvenlik kontrol listesini tamamlayınız: depoda gizli anahtar

4. Örnek üzerinden yavaşça ilerle

Gün 44 • örnek

```
logger.LogInformation(
    "Ticket {TicketId} created by {UserId}",
    ticket.Id,
    userId);
```

5. Bu kodun fikri nedir?

- Structured logging alanların aranmasını kolaylaştırır.
- Log seviyesi doğru seçilmelidir.
- Kişisel ve hassas verinin loglara sızması güvenlik problemidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Loglama ve güvenlik kontrolü ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Kontrol listesi tamamlandı.

Dokümantasyon ve kod incelemesi

BUGÜNÜN HEDEFİ

Yazılan kodun doğruluğunu test etmek, sorunları izlenebilir kılmak ve projeyi belgelemek.

1. Önce konuyu anla

README bir projeyi ilk kez gören kişiye yol gösterir. Kurulum, gerekli araçlar, environment ayarları, Compose, migration, servisler ve ekran görüntüleri kısa ama uygulanabilir olmalıdır. Code review kaliteyi ve bakım maliyetini konuşmak için fırsattır.

GÜNLÜK HAYATTAN BENZETME

Yeni bir eve gelen kişiye elektrik panosunun nerede olduğunu anlatan kullanım kılavuzu gibi düşün.

2. Kaynak programda ne öğrenilecek?

- İyi bir README nasıl olur
- Kod incelemesinden (code review) nasıl

3. Uygulama / ödev

- ☐ README: kurulum, mimari şema, teknolojilerin görevleri, ekran
- ☐ Cuma: mentorla kapsamlı kod incelemesi; notları bir sonraki

4. Örnek üzerinden yavaşça ilerle

Gün 45 • örnek

```
# StajDesk

## Kurulum
1. .env.example'dan yerel ayarları oluşturun.
2. docker compose up --build

## Servisler
- Web
- API
- PostgreSQL
- Redis
- RabbitMQ
```

5. Bu kodun fikri nedir?

- README komutları hangi sırada çalıştıracağını açıklar.
- Mimari şema parçaların görevini gösterir.
- Review notlarını açık issue/task olarak çevirmek sonraki haftayı kolaylaştırır.

Kendine sor

Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Dokümantasyon ve kod incelemesi ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?

Sık yapılan hata

Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

6. Kavrama soruları

- 1 Test yazmak zaman kaybı mıdır? Kendi projenizden bir örnekle cevaplayınız.
- 2 Bir hata canlı ortamda çıksaydı loglarınız sebebini bulmaya yeter miydi?
- 3 Projeyi ilk kez gören biri README ile sistemi çalıştırabilir mi? Lise Stajyer Programı | StajDesk Projesi info@devoplus.com.tr | www.devoplus.com.tr Sayfa 21 FAZ 8 · KAPANIŞ Hafta 16 | Son Dokunuşlar ve Final Sunumu Haftanın hedefi: Projeyi tamamlamak, öğrendiklerinizi ekibe anlatmak ve staj sürecini değerlendirmek.

Eksikleri tamamlama

BUGÜNÜN HEDEFİ

Projeyi tamamlamak, öğrendiklerini ekibe anlatmak ve staj sürecini değerlendirmek.

1. Önce konuyu anla

Son günlerde her işi aynı anda yapmak yerine etkisine göre önceliklendirmek gerekir. Güvenlik, veri kaybı ve ana kullanıcı akışını bozan problemler genellikle düşük etkili görsel ayrıntılardan önce gelir.

GÜNLÜK HAYATTAN BENZETME

Bir binada önce güvenliği tehdit eden problemi çözmek, sonra duvar rengini seçmek gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Önceliklendirme: hangi hata önce düzeltilmeli?

3. Uygulama / ödev

- ☐ Kod incelemesinden gelen notları kapatınız
- ☐ Zaman kalırsa "Bonus Görevler" listesinden birini seçiniz

4. Örnek üzerinden yavaşça ilerle

Gün 46 • örnek

P0: kritik güvenlik / veri kaybı
P1: ana akış bozuluyor
P2: önemli iyileştirme
P3: düşük etki

5. Bu kodun fikri nedir?

- Öncelik seviyeleri ekip içinde değişebilir.
- Etki + risk + efor birlikte değerlendirilir.
- Kod review notlarını kapatmak finalden önce teknik borcu azaltır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? "Eksikleri tamamlama ne işe yarıyor?" sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemler.

GÜN SONU ÇIKTISI

Açık hata listesi boş.

Sunum hazırlığı

BUGÜNÜN HEDEFİ

Projeyi tamamlamak, öğrendiklerini ekibe anlatmak ve staj sürecini değerlendirmek.

1. Önce konuyu anla

Teknik sunumun amacı teknoloji isimlerini saymak değil, problem → çözüm → mimari → nedenler → demo hikâyesini kurmaktır. Teknik olmayan dinleyici “neden buna ihtiyaç var?” sorusunun cevabını almalıdır.

GÜNLÜK HAYATTAN BENZETME

Bir evin içindeki malzemelerin markasını saymak yerine, odaların neden var olduğunu ve nasıl kullanıldığını anlatmak gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Teknik bir konuyu teknik olmayan birine

3. Uygulama / ödev

- ☐ 10-15 slaytlık sunum: problem, mimari, her teknolojinin görevi
- ☐ Mentorla prova yapınız

4. Örnek üzerinden yavaşça ilerle

Gün 47 • örnek

1. Problem
2. StajDesk
3. Mimari
4. Backend / DB
5. Redis / RabbitMQ
6. Frontend
7. Güvenlik / Test
8. Demo
9. Öğrendiklerim

5. Bu kodun fikri nedir?

- Her teknoloji için “şu problemi çözmek için seçtim” cümlesi hazırla.
- Demo gerçek kullanıcı akışını gösterebilir.
- Sunumda ekran görüntüsü gibi yedek materyal bulundurmak iyi hazırlıktır.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Sunum hazırlığı ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Sunum ve demo senaryosu hazır.

Final sunumu ve değerlendirme

BUGÜNÜN HEDEFİ

Projeyi tamamlamak, öğrendiklerini ekibe anlatmak ve staj sürecini değerlendirmek.

1. Önce konuyu anla

Finalde sadece çalışan ürünü göstermek değil, teknik kararlarını ve öğrenme sürecini anlatmak önemlidir. Retrospektif, neyin iyi gittiğini, nerede zorlandığını ve bundan sonra ne öğreneceğini somutlaştırır.

GÜNLÜK HAYATTAN BENZETME

Bir yarıştan sonra sadece dereceye değil, antrenmanda neyin işe yaradığına bakmak gibi düşün.

2. Kaynak programda ne öğrenilecek?

- Geri bildirim almak ve vermek

3. Uygulama / ödev

- ☐ Ekibe 20 dakikalık sunum + soru-cevap
- ☐ Retrospektif: neler iyi gitti, neler zorladı, bundan sonra ne

4. Örnek üzerinden yavaşça ilerle

Gün 48 • örnek

Retrospektif
İyi giden: ...
Zorlayan: ...
Sonraki hedef: ...

5. Bu kodun fikri nedir?

- “Her şeyi öğrendim” yerine somut öğrenme örnekleri ver.
- Sonraki hedef ölçülebilir olsun.
- Geri bildirim savunma değil öğrenme verisidir.

Kendine sor	Sık yapılan hata
Bu konuyu mentora 1-2 dakikada anlatabilir miyim? “Final sunumu ve değerlendirme ne işe yarıyor?” sorusuna kendi cümlemlerle cevap verebiliyor muyum?	Kod çalıştı diye geçme. Bir satırı değiştir, tekrar çalıştır ve sonucun neden değiştiğini gözlemle.

GÜN SONU ÇIKTISI

Staj tamamlandı.

6. Kavrama soruları

- 1 Bu projeyi baştan yapsaydınız neyi farklı yapardınız?
- 2 5 teknolojinin her birini bir cümleyle, teknik olmayan birine anlatınız.
- 3 Bundan sonraki öğrenme hedefiniz nedir?

Başlangıç, Git ve Web'in Çalışma Mantığı - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Tarayıcıya bir adres yazıp Enter'a bastığınızda neler olur?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 1 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Docker ile Tanışma - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Image ile container arasındaki farkı bir benzetmeyle açıklayınız.

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 2 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

C# Temelleri - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 decimal ile double arasındaki fark nedir? Para için hangisi kullanılır, neden?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 3 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Nesne Yönelimli Programlama (OOP) - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Interface kullanmanın bize ne faydası var? ITicketRepository örneği üzerinden anlatınız.

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 4 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

ASP.NET Core Web API - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Uygulamayı yeniden başlattığınızda talepler neden kayboluyor?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 5 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

İlişkisel Veritabanı ve SQL - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Talepleri bir metin dosyasında değil de veritabanında tutmamızın 3 nedeni nedir?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 6 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Entity Framework Core ile API'yi Veritabanına Bağlamak - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Repository'yi değiştirdiğimizde controller'ın değişmemesi neyi kanıtlıyor?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 7 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Doğrulama, Hata Yönetimi ve Kimlik

Doğrulama - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 JWT'nin içeriği okunabiliyorsa neden içine şifre veya gizli bilgi koymayız?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 8 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Frontend Temelleri ve Vue.js'e Giriş - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Reaktivite nedir? Vue olmadan aynı etkiyi nasıl elde ederdiniz?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 9 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Vue Router, Pinia ve API Bağlantısı - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 CORS hatası neden Postman'da değil de tarayıcıda çıkar?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 10 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Uygulama Ekranları - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Uygulamanın şu anki hâlinde performans sorunu çıkabilecek yerler nereler?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 11 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Redis ile Önbellekleme ve Hız Sınırlama - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Redis'i ana veritabanı olarak kullansaydık ne gibi riskler olurdu?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 12 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

RabbitMQ ile Asenkron Mesajlaşma - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Bildirimi doğrudan API içinde göndermek yerine neden kuyruğa atıyoruz?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 13 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Tüm Sistemin Docker ile Çalıştırılması - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Multi-stage build neden image boyutunu küçültür?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 14 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Kalite: Test, Loglama ve Dokümantasyon - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Test yazmak zaman kaybı mıdır? Kendi projenizden bir örnekle cevaplayınız.

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 15 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Son Dokunuşlar ve Final Sunumu - Haftayı kapat

Önce cevapları hafızadan yaz. Sonra önceki günlere dön ve eksiklerini düzelt. Amaç mentora kusursuz cevap vermek değil, düşünceni açıkça anlatabilmektir.

Bu haftanın üç temel sorusu

- 1 Bu projeyi baştan yapsaydınız neyi farklı yapardınız?

Mentor provası	LOG.md için şablon
5 dakika boyunca haftayı anlat: ne öğrendim → ne yaptım → nerede zorlandım → nasıl çözdüm.	# Hafta 16 Bugün öğrendiğim: ... Takıldığım nokta: ... Çözümüm: ... Bir sonraki adım: ...

HAFTAYI GEÇTİM Mİ?

Konuya bakmadan temel kavramları açıklayabiliyor, uygulamayı çalıştırabiliyor ve kod değişikliğini Git ile kaydedebiliyorsan bir sonraki haftaya geçebilirsin.

Artık 48 günü ayrı ayrı değil, birbirine bağlanan bir sistem olarak gör.

Kullanıcı tarayıcıdan giriş yaptığında Vue/axios veya fetch üzerinden API'ye HTTP isteği gönderir. API Controller → Service → Repository/EF Core yoluyla PostgreSQL'e ulaşabilir. Redis tekrar eden okumaları hızlandırabilir veya rate-limit sayaçlarını tutabilir. Ticket oluşturulduğunda API bir event'i RabbitMQ'ya yayınlayabilir; Worker bunu tüketip notification oluşturabilir. Docker Compose bütün servisleri aynı çalışma ortamında birleştirir. Test, log ve README ise bu sistemin güvenilir ve anlaşılır biçimde sürdürülebilmesine yardım eder.

Ana veri yolu	Asenkron yan yol
Browser → Vue → HTTP → API → Controller → Service → Repository/EF Core → PostgreSQL	API → RabbitMQ → Queue → Worker → Notification → PostgreSQL → Vue polling

SON KONTROL

Kendine şu cümleyi tamamlat: “StajDesk'te PostgreSQL kalıcı veri için, Redis hızlı tekrar erişim/counter için, RabbitMQ asenkron mesajlaşma için, Vue kullanıcı arayüzü için, ASP.NET Core API HTTP katmanı için kullanılıyor.”

Resmî doğrulama kaynakları

- Microsoft Learn - ASP.NET Core Dependency Injection: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection>
- Microsoft Learn - EF Core migrations: <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/applying>
- Docker Docs - Compose: <https://docs.docker.com/compose/>
- Docker Docs - Multi-stage builds: <https://docs.docker.com/get-started/docker-concepts/building-images/multi-stage-builds/>
- PostgreSQL Documentation: <https://www.postgresql.org/docs/>
- Vue.js Documentation: <https://vuejs.org/guide/>
- Vue Router Documentation: <https://router.vuejs.org/>
- Pinia Documentation: <https://pinia.vuejs.org/>
- Redis Documentation: <https://redis.io/docs/latest/>
- RabbitMQ Documentation: <https://www.rabbitmq.com/docs/>
- OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>

Bu kitap öğrenme amacıyla hazırlanmıştır. Kurulum komutları ve framework API'leri zaman içinde değişebileceğinden gerçek projede resmi dokümantasyon ve proje sürümü birlikte kontrol edilmelidir.